

UNIVERSIDAD NACIONAL

Programación IV

PROYECTO 1 — PRUEBA DE CONCEPTO (PoC)

Sistema Web de Turismo Local

con Autenticación SSO de Google y Acceso mediante Código QR

Curso: Programación 4

Tipo: Proyecto 1 — Prueba de Concepto (PoC)

Valor: 25 % de la nota del curso

Profesor: Darin Mauricio Gamboa

Modalidad: Trabajo grupal

Fecha de Entrega: 6 de junio

Stack permitido: React + Bootstrap · Spring Boot o Django

Despliegue: Nube pública (preferente Google Cloud Platform)

Repositorio: GitHub con CI/CD mediante GitHub Actions

Documento académico para uso exclusivo del curso

I Ciclo Lectivo · Universidad Nacional

Tabla de Contenido

Tabla de Contenido	3
1. Descripción General	4
2. Justificación Académica	4
3. Objetivos de Aprendizaje	5
3.1 Objetivo general	5
3.2 Objetivos específicos	5
4. Alcance del Proyecto	5
4.1 Dentro del alcance	5
4.2 Fuera del alcance (versión actual)	6
5. Requisitos Técnicos Obligatorios	7
6. Arquitectura Propuesta	8
7. Mockups de las Pantallas Principales	10
7.1 Mockup del código QR físico	10
7.2 Mockup de la pantalla de autenticación	12
7.3 Mockup de la lista de lugares turísticos	13
7.4 Mockup del panel de administración (Bonus)	14
7.5 Mockup del mapa interactivo (Bonus)	15
7.6 Mockup de la página de error	16
7.7 Diagrama de flujo de usuario	17
8. Especificación Funcional	20
8.1 Generación del código QR	20
8.2 Pantalla de autenticación	20
8.3 Pantalla de lugares turísticos	20
8.4 Endpoints mínimos del backend	20
8.5 Persistencia mínima	21
9. Despliegue en la Nube	22
10. Integración y Despliegue Continuo (CI/CD)	23
11. Pruebas Unitarias	23
12. Entregables	24
13. Rúbrica de Evaluación (15 % del curso)	24
14. Cronograma Sugerido	25
15. Reglas y Restricciones	25
16. Recursos Recomendados	25
17. Referencias Bibliográficas	27

1. Descripción General

Se desarrollará una Prueba de Concepto (PoC) and MVP de un sistema web siguiendo una arquitectura cliente-servidor, cuyo dominio de negocio es la promoción turística de un pueblo. El usuario accederá al sistema escaneando un código QR físico (impreso, colocado en un punto turístico, restaurante, hotel u oficina del municipio), el cual lo dirigirá automáticamente a la pantalla de autenticación.

Tras iniciar sesión con su cuenta de Gmail mediante Google SSO (OAuth 2.0), se le presentará una lista de lugares a visitar del pueblo seleccionado. Esta solución busca demostrar la viabilidad técnica de un sistema escalable de turismo local con autenticación federada y acceso contextual basado en ubicación.

El proyecto introduce al estudiante a los principios de desarrollo web full-stack moderno, integración de servicios de identidad federada, despliegue en la nube y prácticas DevOps mediante Integración y Despliegue Continuo (CI/CD).

2. Justificación Académica

El presente proyecto se enmarca dentro de los contenidos del curso Programación 4 de la Universidad Nacional, cuyo objetivo es consolidar competencias de desarrollo de software profesional, incluyendo el manejo de arquitecturas distribuidas, frameworks modernos del lado del cliente y del servidor, integración con servicios externos, automatización de procesos de construcción y despliegue, así como buenas prácticas de verificación mediante pruebas unitarias.

El enfoque de Prueba de Concepto permite que el estudiante valide la viabilidad técnica de la solución antes de comprometerse con el desarrollo completo, replicando un escenario realista de la industria del software.

3. Objetivos de Aprendizaje

3.1 Objetivo general

Construir una aplicación web bajo arquitectura cliente-servidor que integre autenticación federada con Google, despliegue automatizado en la nube y prácticas DevOps mediante CI/CD, aplicando los principios de ingeniería de software estudiados en el curso.

3.2 Objetivos específicos

Al finalizar el proyecto, el estudiante será capaz de:

- **Diseñar e implementar** una aplicación web bajo arquitectura cliente-servidor desacoplada.
- **Construir** una interfaz de usuario en React utilizando un framework CSS (Bootstrap, Tailwind, Material UI u otro).
- **Desarrollar** una API REST en Spring Boot (Java 17) o Django (Python 3.11+).
- **Integrar** autenticación federada con Google mediante OAuth 2.0 / OpenID Connect.
- **Aplicar** persistencia con un motor de base de datos relacional o no relacional desplegado en la nube.
- **Configurar** un pipeline de Integración y Despliegue Continuo (CI/CD) con GitHub Actions.
- **Generar y leer** códigos QR como mecanismo de acceso al sistema.
- **Desplegar** la aplicación en la nube, preferentemente en Google Cloud Platform.
- **Escribir y ejecutar** pruebas unitarias verificables tanto en frontend como en backend.

4. Alcance del Proyecto

4.1 Dentro del alcance

- Pantalla de autenticación con Google SSO (cuentas de Gmail).
- Generación y lectura de códigos QR vinculados a un pueblo o punto turístico.
- Listado de lugares a visitar por pueblo, con información mínima: imagen, nombre, descripción, categoría y dirección.
- Backend REST con endpoints documentados.
- Persistencia en una base de datos en la nube.
- Pipeline CI/CD funcional con GitHub Actions.

- Despliegue público accesible mediante URL.

4.2 Fuera del alcance (versión actual)

- Aplicación móvil nativa (iOS/Android).
- Inteligencia artificial para recomendaciones personalizadas.
- Funcionalidades sociales (comentarios, reseñas verificadas, redes).
- Continuous inspection.

5. Requisitos Técnicos Obligatorios

Categoría	Tecnología requerida
Frontend	React + Bootstrap u otro framework CSS
Backend	Spring Boot (Java 17) o Django (Python 3.11+)
Arquitectura	Cliente-Servidor desacoplado, REST API + JSON
Autenticación	Google SSO mediante OAuth 2.0 / OpenID Connect
Base de datos	A elección del equipo (PostgreSQL, MySQL, MongoDB, Firestore, etc.)
QR	Generación y escaneo (qrcode, react-qrcode, ZXing)
Pruebas	Base mínima de pruebas unitarias en frontend y backend
CI/CD	GitHub Actions con etapas de build, test y deploy
Despliegue	Nube pública (preferente GCP — Cloud Run, App Engine)
Repositorio	GitHub con flujo de ramas profesional

6. Arquitectura Propuesta

La arquitectura del sistema sigue el modelo cliente-servidor desacoplado, donde el frontend y el backend se comunican exclusivamente mediante una API REST sobre HTTPS. La autenticación se delega al proveedor de identidad de Google. El diagrama siguiente sintetiza los componentes principales y su flujo de información:



7. Mockups de las Pantallas Principales

A continuación se presentan los mockups visuales del flujo principal de la aplicación: el código QR físico que actúa como punto de entrada, la pantalla de autenticación con Google y el listado de lugares turísticos del pueblo. Estos mockups sirven como referencia de diseño y experiencia de usuario para los equipos de desarrollo, sin restringir la creatividad visual de cada solución.

7.1 Mockup del código QR físico

Cada pueblo participante contará con un cartel impreso que muestra el código QR único de acceso. Al escanearlo, el dispositivo redirige al navegador del usuario hacia la URL del frontend, transmitiendo como parámetro el identificador del pueblo. La Figura 1 ilustra el formato esperado del cartel de acceso.



Figura 1. Mockup del código QR físico de acceso al sistema.

7.2 Mockup de la pantalla de autenticación

Al ingresar al sistema, el usuario debe iniciar sesión con su cuenta de Gmail mediante el flujo OAuth 2.0 de Google. La pantalla muestra el nombre del pueblo identificado por el QR y el botón de autenticación federada. La Figura 2 representa el diseño esperado de esta vista.

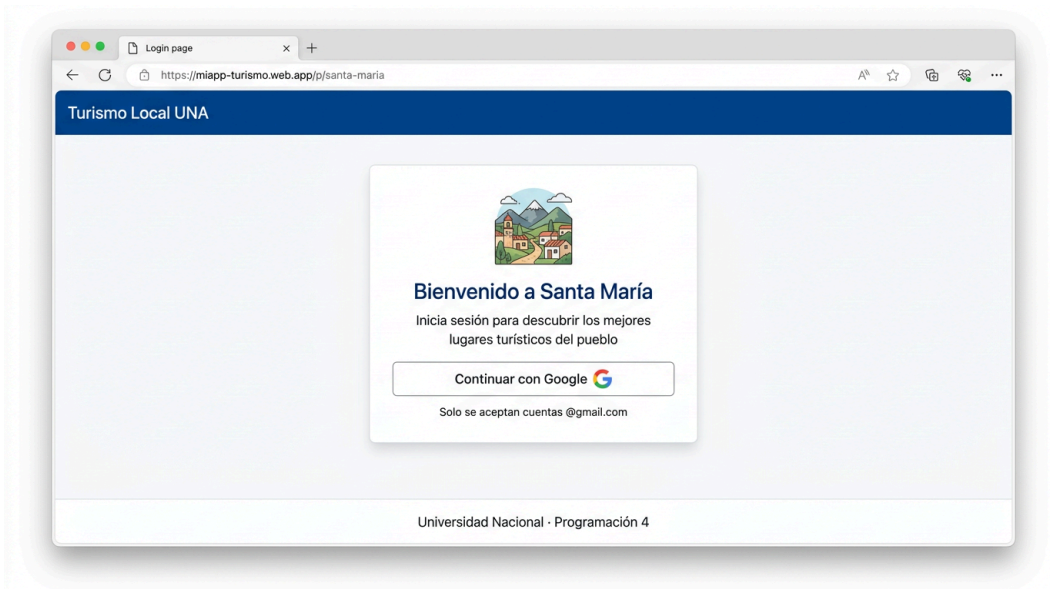


Figura 2. Mockup de la pantalla de autenticación con SSO de Google.

7.3 Mockup de la lista de lugares turísticos

Tras la autenticación exitosa, el usuario accede a la lista de lugares a visitar del pueblo correspondiente. Cada tarjeta presenta una imagen representativa, el nombre del lugar, la categoría (mirador, cultural, gastronomía, etc.), una breve descripción y la dirección referencial. La Figura 3 ilustra el resultado esperado.

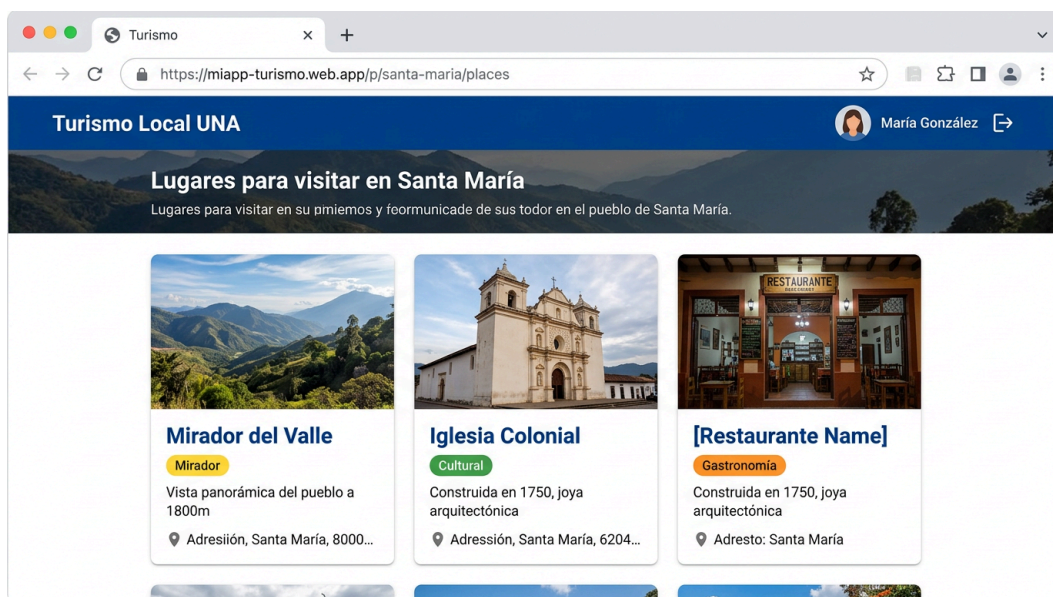


Figura 3. Mockup de la pantalla de lista de lugares turísticos.

7.4 Mockup del panel de administración (Bonus)

Como funcionalidad adicional valorada con puntos extras (ver Rúbrica, sección 13), el equipo puede implementar un panel de administración protegido para la gestión CRUD de pueblos y lugares turísticos. Este módulo está restringido a usuarios con rol administrador y permite agregar, editar, activar/desactivar o eliminar registros del catálogo. La Figura 4 ilustra el diseño esperado del panel administrativo.

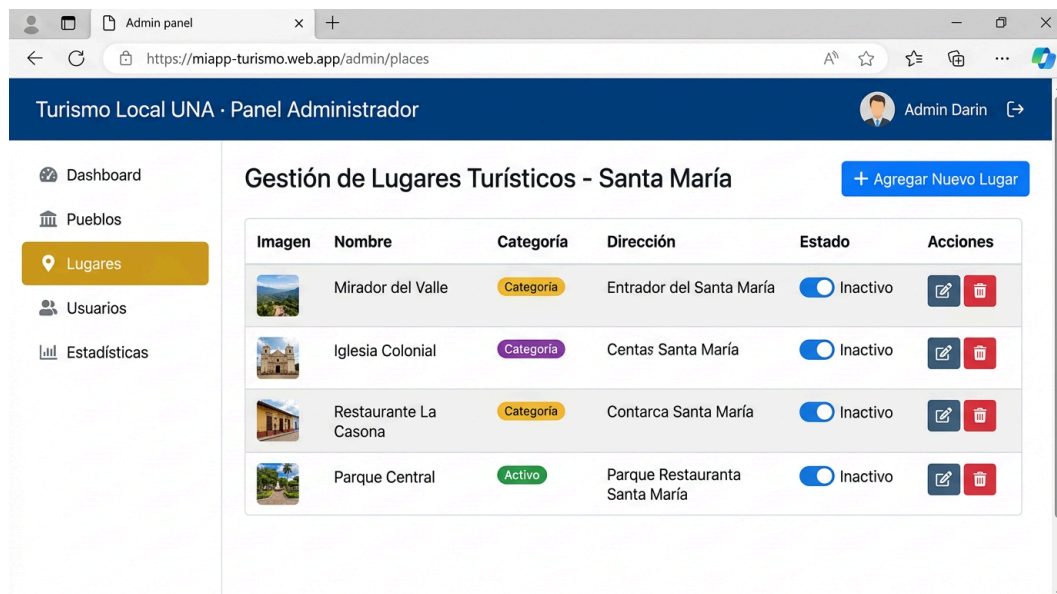


Figura 4. Mockup del panel de administración con CRUD de lugares turísticos.

7.5 Mockup del mapa interactivo (Bonus)

Otra funcionalidad bonus consiste en visualizar los lugares turísticos sobre un mapa interactivo con marcadores geolocalizados. Cada marcador representa un lugar mediante un color distintivo según su categoría (mirador, cultural, gastronomía, parque) y permite consultar un resumen emergente con foto, descripción breve y enlace al detalle. Se sugiere el uso de bibliotecas como Leaflet, Google Maps JavaScript API o Mapbox GL. La Figura 5 representa la vista cartográfica esperada.

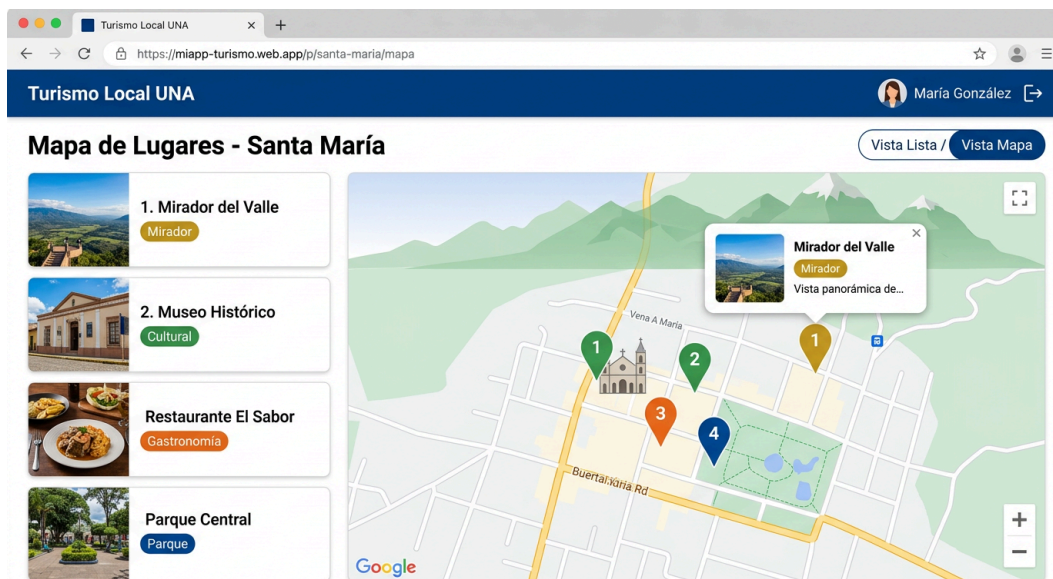


Figura 5. Mockup del mapa interactivo con marcadores de lugares turísticos.

7.6 Mockup de la página de error

El sistema debe contemplar el manejo gracioso de errores cuando un código QR sea inválido, el pueblo no exista en la base de datos, la sesión haya expirado o no se cuente con conexión a internet. La página de error debe ser informativa, mostrar las posibles causas y ofrecer acciones de recuperación al usuario (volver al inicio o reintentar). La Figura 6 ilustra el diseño recomendado para esta vista.

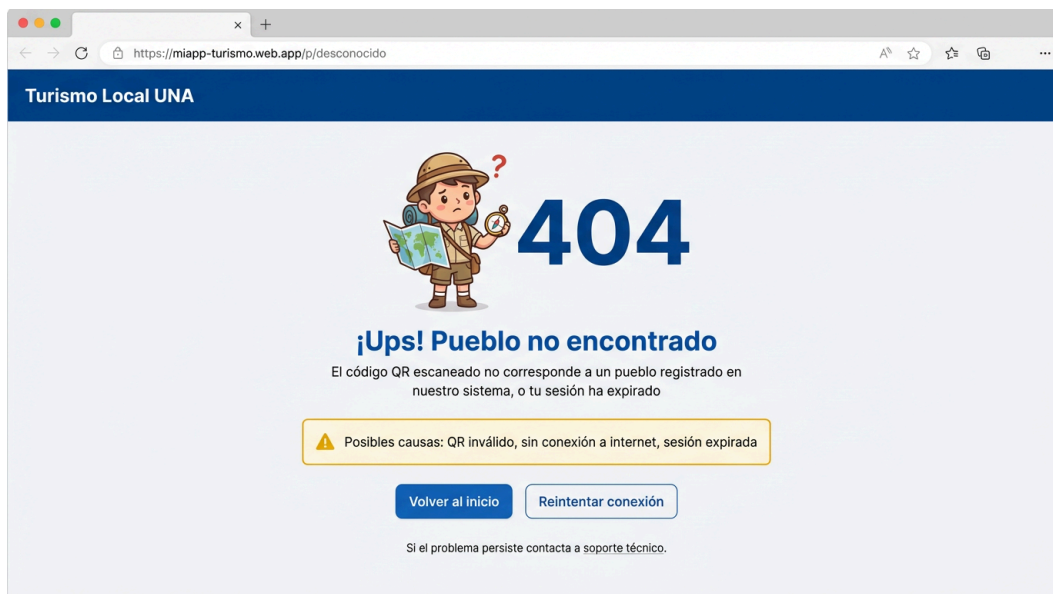


Figura 6. Mockup de la página de error 404 con opciones de recuperación.

7.7 Diagrama de flujo de usuario

Para complementar la comprensión integral del sistema, la Figura 7 presenta el diagrama de flujo de usuario que conecta todas las pantallas descritas previamente. El diagrama parte del momento en que el usuario escanea el código QR físico (Figura 1) y muestra los caminos posibles según la validez de la URL, la autenticación con Google y el rol asignado al usuario.

El flujo cliente conduce al listado de lugares turísticos y, opcionalmente, al mapa interactivo (bonus). El flujo administrador deriva al panel de gestión CRUD (bonus) con interacción directa hacia la base de datos en la nube. Cualquier ruta inválida es interceptada por la página de error 404. El código de colores diferencia: azul para flujo principal, dorado para acciones administrativas o bonus, verde para caminos exitosos y rojo para errores.

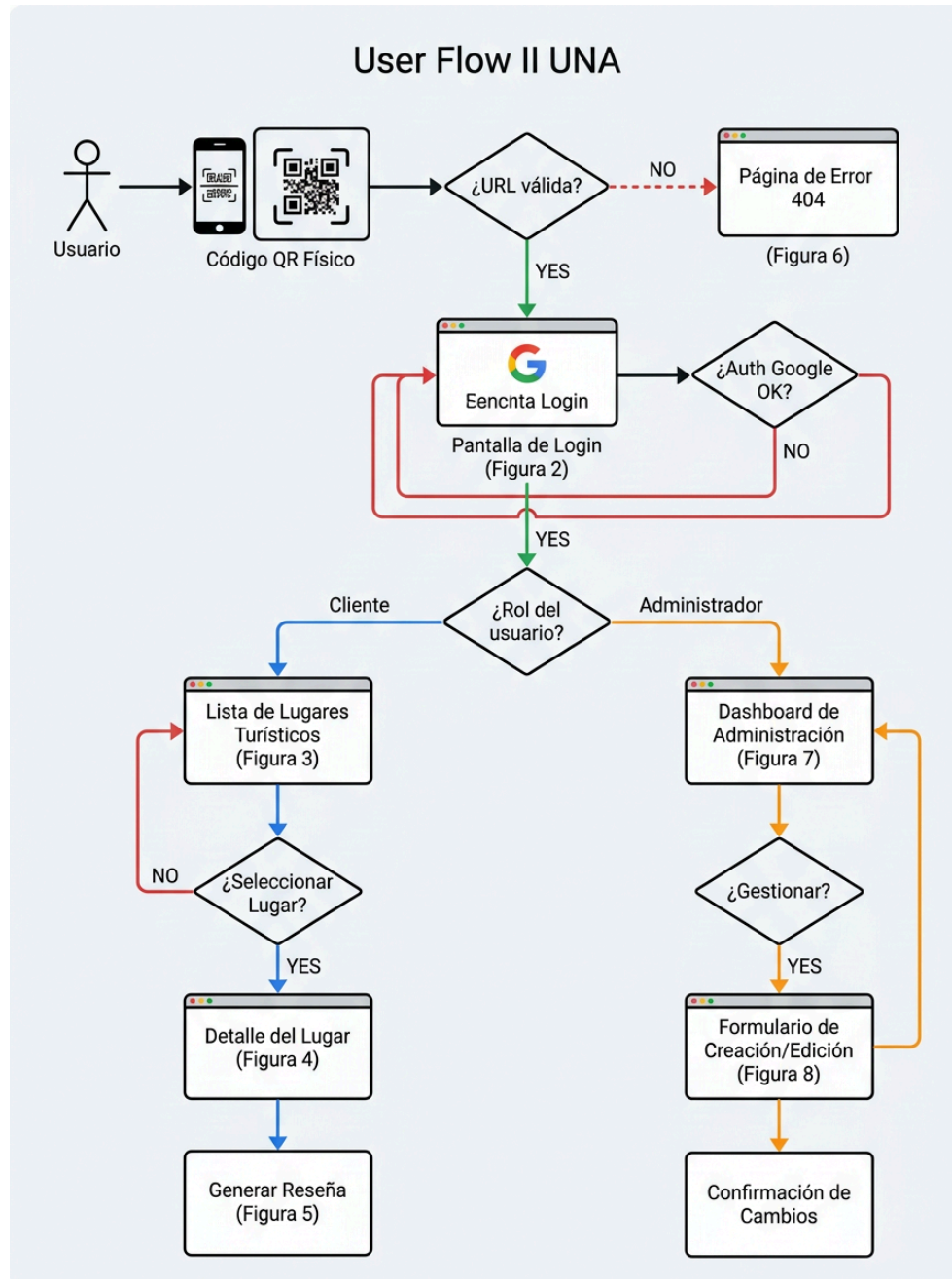


Figura 7. Diagrama de flujo de usuario que conecta todas las pantallas del sistema.

Resultado esperado del proyecto

El equipo deberá lograr un flujo funcional similar al ilustrado en las Figuras 1, 2 y 3, con datos reales del pueblo asignado, QR escaneable desde un dispositivo físico y autenticación verificada con cuenta de Gmail real. Las Figuras 4 y 5 corresponden a funcionalidades bonus que otorgan puntos extras, mientras

que la Figura 6 representa el manejo obligatorio de errores del sistema. La Figura 7 sintetiza el flujo de usuario end-to-end y debe usarse como guía de navegación al implementar la solución.

8. Especificación Funcional

8.1 Generación del código QR

- El sistema (o el equipo manualmente) debe generar un **código QR único por pueblo** o por punto turístico.
- El QR debe codificar una URL del frontend con un parámetro identificador, por ejemplo:
`https://miapp-turismo.web.app/p/santa-maria`
- Debe documentarse un endpoint o pantalla de administración para regenerar QR (al menos manualmente).

8.2 Pantalla de autenticación

- Al escanear el QR, el usuario llega a una pantalla limpia que muestra el nombre del pueblo identificado por el parámetro de la URL.
- Botón **Continuar con Google** que dispara el flujo OAuth 2.0.
- Solo se aceptan correos de Gmail (@gmail.com).
- La sesión se mantiene activa mediante token JWT o cookie segura.

8.3 Pantalla de lugares turísticos

Tras autenticarse, el usuario verá una lista de lugares a visitar del pueblo correspondiente. Cada elemento de la lista debe contener como mínimo:

- Imagen representativa del lugar.
- Nombre del lugar.
- Breve descripción.
- Categoría (restaurante, parque, museo, mirador, etc.).
- Dirección o ubicación referencial.

Si el equipo desea agregar mapa con marcadores, calificaciones o filtros, sumará puntos extra (ver Rúbrica).

8.4 Endpoints mínimos del backend

Método	Endpoint	Descripción
POST	/api/auth/google	Verifica el ID Token de Google y emite la sesión

GET	/api/towns/{id}	Retorna los datos del pueblo identificado
GET	/api/towns/{id}/places	Lista los lugares del pueblo
GET	/api/users/me	Información del usuario autenticado

8.5 Persistencia mínima

Tablas o colecciones sugeridas: users, towns, places. El equipo es libre de agregar más entidades según sea necesario.

9. Despliegue en la Nube

El sistema debe estar públicamente accesible mediante una URL durante la presentación final. Se recomienda utilizar Google Cloud Platform por su integración natural con OAuth 2.0 y por la simplicidad académica de sus servicios serverless.

Componente	Servicio sugerido (GCP)
Frontend React	Firebase Hosting o Cloud Run
Backend Spring Boot/Django	Cloud Run o App Engine
Base de datos	Cloud SQL (PostgreSQL/MySQL) o Firestore
Imágenes	Cloud Storage o URLs externas

Nubes alternativas aceptadas

AWS, Azure, Render, Railway o Fly.io son aceptadas como nubes alternativas. Sin embargo, Google Cloud Platform es la opción preferente por su integración directa con OAuth 2.0 y por la facilidad de gestionar servicios académicos sin costo dentro del tier gratuito.

10. Integración y Despliegue Continuo (CI/CD)

El equipo debe configurar un archivo `.github/workflows/ci-cd.yml` que automatice las siguientes etapas en cada push a las ramas `main` y `develop`:

- **Build** del frontend (`npm run build`).
- **Build** del backend (`mvn package` o `python -m build`).
- **Pruebas unitarias** (frontend y backend).
- **Linter / análisis estático** (opcional, +5 puntos).
- **Despliegue automático** a la nube en la rama `main`.

Gestión de credenciales

Las credenciales sensibles (claves de Google, secretos OAuth, llaves de servicio GCP, contraseñas de base de datos) deben gestionarse mediante GitHub Secrets. Bajo ninguna circunstancia deben commitarse al repositorio.

11. Pruebas Unitarias

Mínimo requerido:

- **Frontend:** al menos 3 pruebas con Jest + React Testing Library (renderizado, navegación, manejo de estado).
- **Backend:** al menos 3 pruebas con JUnit (Spring Boot) o pytest (Django) sobre controladores y servicios.
- Las pruebas deben **ejecutarse automáticamente** en el pipeline CI.
- Cobertura mínima recomendada: **50%** sobre las clases con lógica de negocio.

12. Entregables

El equipo deberá entregar los siguientes artefactos:

- **Repositorio GitHub** con código fuente completo, organizado en carpetas frontend/ y backend/.
- **README.md** con descripción, stack utilizado, instrucciones para correr localmente, URL desplegada, capturas y QR de prueba.
- **Documento técnico** (PDF, máximo 8 páginas): arquitectura, modelo de datos, flujo de autenticación, capturas de la app y del pipeline CI/CD.
- **Aplicación desplegada y funcional** durante la presentación.
- **Código QR físico o impreso** que será escaneado en clase para validar el flujo completo.
- **Presentación oral** de 10 a 15 minutos por equipo.

13. Rúbrica de Evaluación (25 % del curso)

Criterio	Puntaje
Frontend en React + Bootstrap correctamente implementado	12 %
Backend en Spring Boot o Django con API REST funcional	15 %
Arquitectura cliente-servidor desacoplada y documentada	8 %
Autenticación SSO con Google funcionando	12 %
Lectura del QR + flujo hasta la lista de lugares	10 %
Persistencia con base de datos en la nube	8 %
Despliegue en la nube (URL pública accesible)	10 %
Pipeline CI/CD con GitHub Actions completo	10 %
Pruebas unitarias (frontend + backend)	8 %
Documentación técnica y README	4 %
Presentación oral y demostración en vivo	3 %
TOTAL	100 %
★ Bonus — Mapa interactivo con marcadores	+5
★ Bonus — Filtros, búsqueda o categorías	+5
★ Bonus — Rol administrador (CRUD de lugares)	+10
★ Bonus — Análisis estático con SonarCloud	+5

14. Cronograma Sugerido

Semana	Hito
1	Configuración del repositorio, Spring Initializr / Django start, React init, definición del modelo
2	Login con Google + estructura del backend + esquema de base de datos
3	Pantallas frontend, integración con backend, manejo de QR
4	Pruebas unitarias + despliegue inicial en la nube
5	CI/CD completo + documentación + presentación final

15. Reglas y Restricciones

- El proyecto debe ser **original**. Se aplicarán las políticas de la Universidad Nacional contra el plagio.
- El uso de IA generativa (ChatGPT, Copilot, Claude, Gemini) está permitido, pero el equipo **debe poder defender línea por línea** cualquier código presentado.
- El flujo Git debe ser profesional: ramas por feature, pull requests y revisiones cruzadas entre los miembros.
- Las credenciales sensibles **jamás** deben commitearse al repositorio.
- La aplicación debe estar **accesible públicamente** durante la defensa final.
- Cada miembro del equipo debe poder explicar cualquier parte del sistema; la defensa puede tornarse individual a criterio del profesor.
- La fecha y hora de entrega son inamovibles, salvo casos justificados según el reglamento universitario.

16. Recursos Recomendados

Documentación oficial y recursos académicos sugeridos:

- **React:** <https://react.dev/learn>
- **Bootstrap:** <https://getbootstrap.com/>
- **Spring Boot:** <https://spring.io/projects/spring-boot>
- **Django REST Framework:** <https://www.django-rest-framework.org/>
- **Google Identity (OAuth 2.0):** <https://developers.google.com/identity/protocols/oauth2>

- **GitHub Actions:** <https://docs.github.com/en/actions>
- **Google Cloud Run:** <https://cloud.google.com/run/docs>
- **Firebase Hosting:** <https://firebase.google.com/docs/hosting>
- **QR (frontend):** <https://www.npmjs.com/package/react-qr-reader>
- **QR (backend):** <https://github.com/zxing/zxing>

17. Referencias Bibliográficas

Formato APA, 7.ª edición.

- Banks, A., & Porcello, E. (2020). Learning React: Modern patterns for developing React apps (2.ª ed.). O'Reilly Media.
- Django Software Foundation. (2025). Django documentation. <https://docs.djangoproject.com/>
- Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Tesis doctoral, University of California, Irvine].
<https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- GitHub. (2025). GitHub Actions documentation. <https://docs.github.com/en/actions>
- Google. (2025). Google Cloud Platform documentation. <https://cloud.google.com/docs>
- Google Identity. (2025). Using OAuth 2.0 for web server applications.
<https://developers.google.com/identity/protocols/oauth2/web-server>
- Hardt, D. (Ed.). (2012). The OAuth 2.0 authorization framework (RFC 6749). Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc6749>
- Mozilla Developer Network. (2025). HTTP overview.
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview>
- Otto, M., & Thornton, J. (2025). Bootstrap 5 documentation. <https://getbootstrap.com/docs/>
- Sakimura, N., Bradley, J., Jones, M., de Medeiros, B., & Mortimore, C. (2014). OpenID Connect Core 1.0. OpenID Foundation. https://openid.net/specs/openid-connect-core-1_0.html
- Spring. (2025). Spring Boot reference documentation. VMware.
<https://docs.spring.io/spring-boot/index.html>
- Walls, C. (2022). Spring in action (6.ª ed.). Manning Publications.

© Documento elaborado para fines académicos por Darin Mauricio Gamboa, profesor del curso Programación 4 — Universidad Nacional. Distribución limitada al curso correspondiente.